GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Modules and requirements

Instructor: Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Abdullatif Yılmaz	230104004031
Hasan Alperen Elmas	1801042604	Batuhan Yanar	220104004059
Ayşe Poyraz	230104004085	Berk Andaçgil	230104004066
Hasan Hüseyin Özdemir	230104004048	Muhib Taha Boy	230104004165
Selim Barut	220104004046	Umutcan Ocak	220104004903

1. Project overview

Çılgın Kurye is a multiplayer motorcycle courier game. Players play it in the web browser of their own phones, so there is nothing to install. Up to 8 players play at the same time in 4 teams, and the whole game runs on a server on one computer. Each player rides a motorcycle in a 3D city and carries packages from the team's store to delivery points. The team with the most points after 5 minutes wins.

The special part of the game is that the phone has no map. To find the delivery point, players must look at the 60 × 60 cm city board on the table. LED strips in the roads of the board show all motorcycles, delivery points and obstacles in real time. The players can also hit the board with their hand to start an earthquake in the game. The admin computer is the control centre. During the game the admin changes parameters and sees each change in three places: in the 3D city on the phones, on the LED board and in live graphs. The computer, the phones and the board talk over Wi-Fi.



Figure 1: The target system: the LED city board, the admin screen and the game on a phone.

2. System architecture

The main computer runs one server program. Inside it, M1 runs the game, M3 moves the motorcycles, M4 runs the traffic and the bots, and M2 does all the network work. This program makes every final decision, so all players see the same game. The phones (M5) only send their buttons and draw the 3D city. The admin page (M6) sends commands and shows graphs. In the LED board, the ESP32 runs the link and the sensor (M7) on one core and the LED engine (M8) on the other core.

We chose the protocol of each link with care. A web browser can only open a WebSocket for two-way talk, so the phones and the admin page use WebSocket, which runs on TCP. The LED board uses two channels. Messages that must never be lost go on TCP: the config, the vibrate commands for the motors, the earthquake events and the status report. Data that comes many times per second goes on UDP: the board state, the sensor samples and the delay probes. With UDP a lost packet is simply

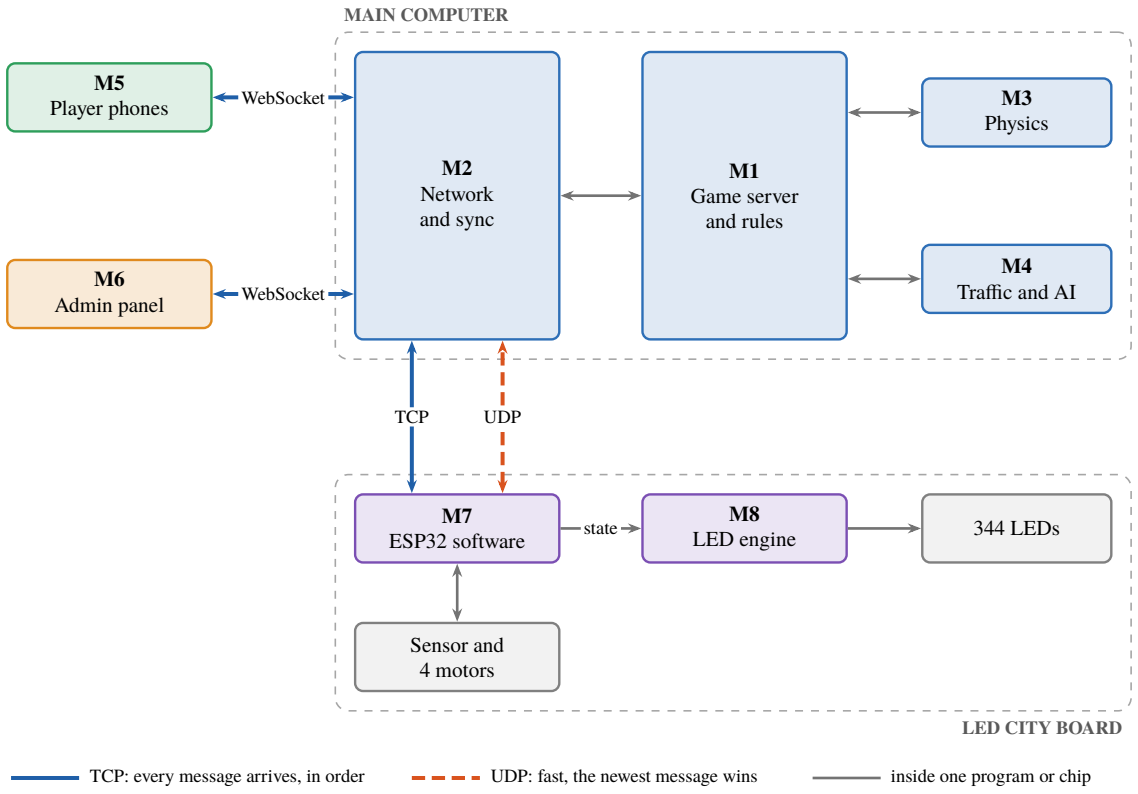


Figure 2: The eight modules and the links between them.

replaced by the next one, so the LEDs never stop to wait.

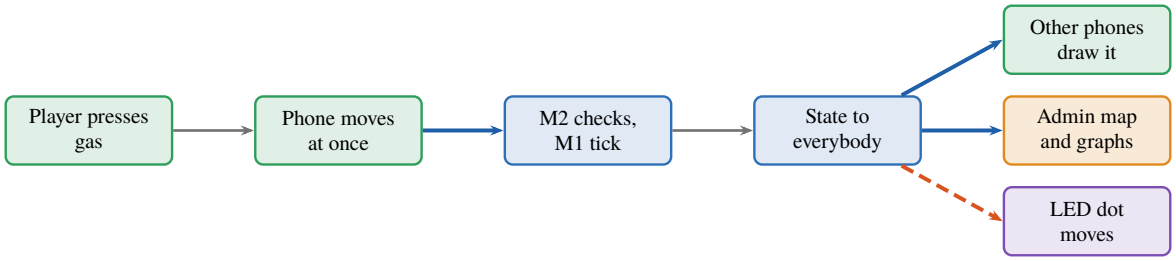


Figure 3: One button press from the phone to every screen and to the board. Blue solid is TCP, orange dashed is UDP.

Five rules hold for the whole system. First, there is one authority: only M1 changes the game state, and only inside a tick. Second, inputs are safe: phones send controls, never positions or scores, and M2 checks every message before M1 sees it. Third, all modules use shared contract files for the messages and one shared city plan, so the 3D city and the board always match. Fourth, each module can be built and tested alone with fake data from its neighbours. Fifth, the inputs of every tick go into a log, so a game can be played again exactly for testing.

3. System requirements

The table lists the requirements of the whole system. The last column shows the modules that meet each requirement.

No	Requirement	Description	Modules
R1	Multiplayer game	At most 8 players in 4 teams (at most 2 per team); a game lasts at most 5 minutes. AI bots fill empty places; the team with the most delivery points wins.	M1 M2 M4 M5
R2	Delivery cycle	The player takes a package from the store; the server chooses a delivery point by travel time with the current obstacles; the point blinks on the board in the player's tone. The player delivers and returns to the store. The near / far chance is set per player by the admin.	M1 M4 M5 M8
R3	Driving and controls	Free driving, lane changes and passing obstacles; no reverse, U-turn by steering. Throttle, brake, left and right buttons on the phone.	M3 M5
R4	Collisions, falls, traffic	In a fast collision both players fall and wait a short penalty. NPC cars create real traffic; hitting a car, a building or an obstacle fast makes the player fall. Penalty time and traffic / obstacle density are admin parameters.	M1 M3 M4

No	Requirement	Description	Modules
R5	No map on the phone	The phone shows only a compass arrow to the delivery point (or back to the store); no distance and no roads. Players must look at the board. "Find me" lights the player's LED white for 1 s.	M1 M5 M8
R6	Obstacles	Four types: heavy traffic, road work, accident, water puddle. On the board red always means "this road slows you down"; the pattern shows the type. The admin can change the effect of each type.	M1 M3 M4 M6 M8
R7	Earthquake mode	Starts by hitting the board by hand or with the admin button (vibration motors under the board); the sensor measures strength and approximate area. In the game the camera shakes, motorcycles fall and roads near the centre close; a shock wave spreads on the board. The strength is an admin parameter.	M1 M3 M5 M6 M7 M8
R8	Physical LED board	60 × 60 cm, 6 roads in each direction (5 × 5 blocks), addressable LEDs on all roads, 3D-printed buildings, four stores with two RGB LEDs each in the corners; the ESP32 connects over Wi-Fi. The 3D city and the board are built from the same city plan.	M5 M7 M8
R9	Live admin panel	All parameters are changed during the game; the admin sees the change in three places: 3D simulation, LED board and live graphs.	M1 M2 M6
R10	Network and measurement	Computer, phones and board talk over Wi-Fi. The admin can add delay, packet loss and a frame-rate limit per player, and delay and loss on the board link; ping, packet loss and frame rate are measured live.	M2 M5 M6 M7
R11	Engineering requirements	The main computer makes all game decisions (single authority); the phone only sends controls and sees its own motorcycle at once (prediction). Inputs and outputs of every module are defined, and each module can be developed and tested with mock data.	M1–M8

4. Modules

The project has eight modules on three sides: the server (M1–M4), the phone and admin pages (M5, M6) and the physical board (M7, M8). Every module has at least 3 members, and every member works in 3 modules. So a module can go on even if one member leaves. An X in the second table means that the member works in that module.

Module	Name	Task	People
M1	Game Server and Rules	Single authority: game loop, rules, score, delivery, fall penalty, admin commands.	5
M2	Network and Synchronization	Phone, admin and board connections; message encoding; safe game state; ping and network delay / loss settings.	4
M3	Physics and Collision	Motorcycle motion, collisions, falls; effects of obstacles and earthquakes; prediction on the phone.	4
M4	Traffic and AI	Routes and travel times, NPC traffic, bot drivers.	3
M5	Player Client and 3D City	3D city on the phone, controls, compass, prediction and correction.	3
M6	Admin Panel and Monitoring	Live parameters, map and obstacle tools, earthquake panel, graphs.	3
M7	ESP32 Software and Communication	Board connection, motion sensor and earthquake detection, vibration motors.	4
M8	LED Animation and Physical Board	LED animation engine and the physical board: base, LED strips, 3D-printed city.	4

Member	M1	M2	M3	M4	M5	M6	M7	M8
Tuğberk Akbulut	X	X					X	
Abdullatif Yılmaz	X		X		X			
Hasan Alperen Elmas			X	X			X	
Batuhan Yanar			X	X		X		
Ayşe Poyraz		X				X		X
Berk Andaçgil	X				X	X		
Hasan Hüseyin Özdemir		X					X	X
Muhib Taha Boy				X			X	X
Selim Barut	X		X		X			
Umutcan Ocak	X	X						X
People	**5**	**4**	**4**	**3**	**3**	**3**	**4**	**4**

5. M1 Game Server and Rules

Team: Tuğberk Akbulut, Abdullatif Yılmaz, Berk Andaçgil, Selim Barut, Umutcan Ocak

5.1 Purpose

M1 is the brain of the game. It runs on the main computer and it is the only part that can change the game. It keeps the game clock and the one true game state. It knows every team, store, package, delivery point, score, fall, obstacle and earthquake. Phones only send their buttons. M1 decides what happens, and then everybody sees the same result.

Runs on	Main computer, in one server program together with M2, M3 and M4
Technology	TypeScript on Node.js; fixed-step game loop at 30 ticks per second; seeded random numbers; JSON files for the city plan and the game log
Requirements	R1 R2 R4 R5 R6 R7 R9 R11

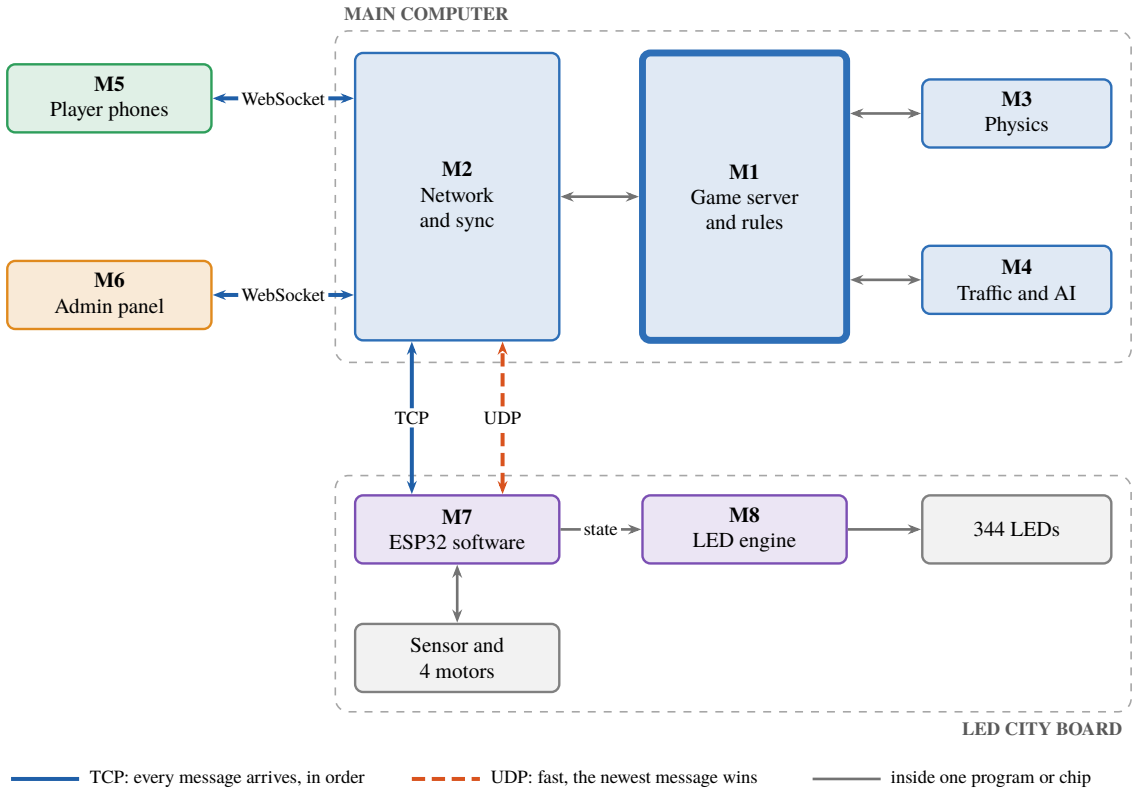


Figure 4: Where M1 sits in the system. The thick box is M1.

5.2 Requirements covered

ID	Requirement	What M1 does for it
R1	Multiplayer game	Lobby with 4 teams and at most 2 players in a team; bots fill empty places; a game is at most 5 minutes; the team with the most points wins
R2	Delivery cycle	Automatic pickup at the store; picks a delivery point by travel time; near or far chance per player
R4	Collisions and falls	Gives a penalty time after every fall that M3 reports
R5	No map on the phone	Puts the compass target and the “Find me” flag into the game state
R6	Obstacles	Keeps the obstacle list that M3, M4, M5 and M8 use
R7	Earthquake	One path for a hand hit, the motors and the admin button; road work and falls inside the radius
R9	Live admin panel	Applies admin commands and sends each parameter to the module that uses it
R11	Single authority	Only M1 changes the game state; every tick can be played again from the log

5.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Controls	M2, from phones	Throttle and brake 0–1, steer −1 to 1, deliver; the newest one in each tick
In	Join, “Find me”	M2, from phones	Player key, name, team; “Find me” request
In	Admin commands	M2, from admin	Parameter changes, start, pause, end, obstacles, earthquake

Dir.	Data	From / to	Content
In	Board events	M2, from board	Earthquake strength 0–10, centre, zone, confidence
In	Traffic	M4	NPC cars, bot controls, travel-time table
In	Physics	M3	New motorcycle positions, collision and fall events
Out	Game state	M2, to phones and admin	Every tick: phase, time, scores, players, NPC cars, obstacles, earthquake
Out	Board state	M2, to board	Players, delivery points, obstacles, earthquake in board coordinates
Out	Metrics	M2, to admin	Twice per second: the values of all admin graphs
Out	Game log	Disk	The inputs of every tick, one JSON line per tick

5.4 Main components

Component	What it does	Related modules
Game loop	Runs the steps of a tick in a fixed order, 30 times per second	M3, M4
Match and lobby	Game phases, teams, bots, keeps a place for 30 s after a player drops	M2, M4
Delivery and scoring	Pickup, delivery check with a reason when it says no, choice of the delivery point, points	M4, M5, M8
Penalties	Penalty time after a fall; asks M3 to stand the motorcycle up again	M3
Obstacles and earthquake	Obstacle list with a version number; earthquake strength, radius, road work and falls	M3, M4, M7
Admin and parameters	Checks commands, keeps every parameter with its limits, sends each group to its owner	M2, M3, M4, M6
State builder and recorder	Builds the frozen game state, the board state and the metrics; writes and replays the game log	M2

5.5 How it works

The tick. The game moves in small steps called ticks. There are 30 ticks in one second, so one tick has about 33 ms. A tick always runs the same eight steps in the same order, as the figure below shows. First M1 takes the inputs that M2 has collected since the last tick. If a phone sent more than one control message, only the newest one is used. Then M1 applies admin commands and board events, updates the game phase, asks M4 for traffic and bot controls, and asks M3 to move all motorcycles. After that the game rules run. At the end M1 builds a frozen copy of the state for M2 and writes one line to the game log. Because the order never changes and all random numbers come from one seed, the same inputs always give the same game.

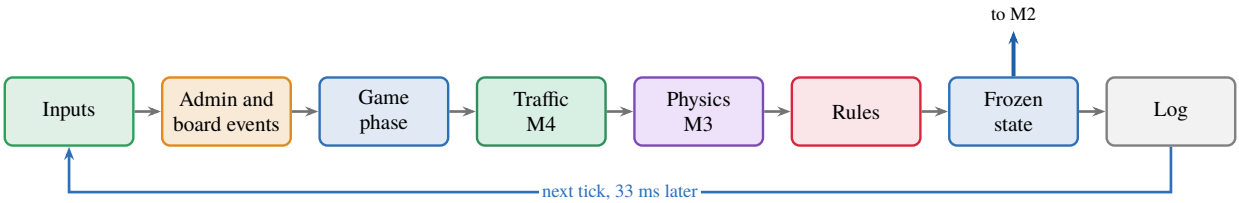


Figure 5: The eight steps of one tick.

Game flow. A game has five phases. In the lobby players join and choose a team. When the admin presses start, a 3 second countdown begins and nobody can move. Then the game runs for at most 5 minutes. The admin can pause and resume the game at any time. When the time ends, M1 shows the final scores and the game is finished. If a phone loses its connection, M1 keeps that player's place for 30 seconds. If the player does not come back, a bot takes the motorcycle and continues.

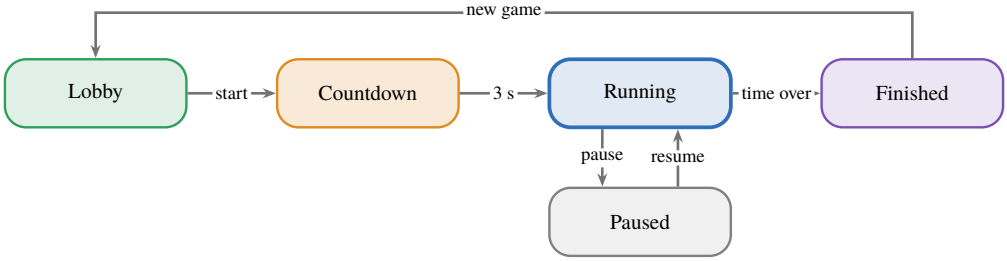
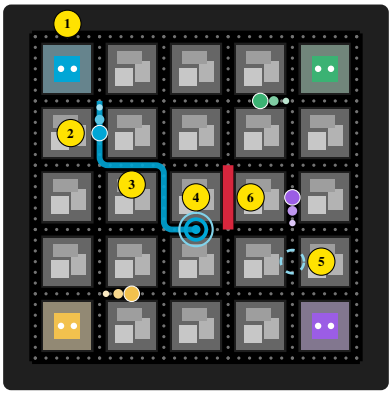


Figure 6: Phases of one game.

Deliveries and points. A player gets a package automatically when the motorcycle is inside the area of the own team's store. M1 then picks a delivery point for this player. M4 gives a table with the travel time from the store to every possible point. The times also count the obstacles that are on the roads at that moment. M1 sorts this list and cuts it in the middle, so it gets a near

half and a far half. The admin sets a “near chance” for each player, and M1 uses it to choose the half. A far point gives more points: points are a base value plus a bonus for each second of the expected travel time. Both numbers are admin parameters. To deliver, the player presses “Deliver”. The button is always active. M1 accepts it only inside the delivery radius and below the speed limit. If M1 says no, it sends the reason, for example “too fast” or “too far”. A fall does not lose the package; the player only waits for the penalty time.



1 Team store, two LEDs in team colour
2 Player with a package, dot with a tail
3 Fastest route found by M4
4 Near delivery point, blinks in team colour
5 A far point, worth more points
6 Road work, so the route goes around

Figure 7: One delivery as it will look on the LED board. The phone shows only a compass arrow.

Obstacles and earthquakes. M1 keeps one list of obstacles. Each obstacle has a type, a road segment, a lane and a position. The list has a version number, and the number goes up on every change. So M4 can see when it must compute routes again, and M5 and M8 can see when they must draw again. All earthquakes take one path in M1, so a hand hit on the board, the motors and the admin button act in the same way. The game strength is the measured strength times an admin factor. Roads inside the earthquake radius get road work, and motorcycles inside the radius may fall.

Admin parameters. Every parameter has a name, a group, a minimum, a maximum and an owner module. When the admin changes a value, M1 checks the limits first. Then it saves the value, sends it to the owner (for example grip to M3 or traffic density to M4) and sends “ok” back to the panel. A wrong value gets an error and the old value stays. Twice per second M1 also puts all graph values (scores, delivery times, speeds, falls, traffic) into one metrics message.

5.6 Testing and demonstration

M1 has no network code. So we can test every rule with fake M2, M3 and M4 parts that run inside the test. The tests check the game flow, the lobby and the bots, the delivery cycle, the fair choice of points, every admin command and the earthquake. One test plays a recorded game again with the same seed and checks that the result is the same. In the demonstration, a full game with bots runs on the main computer. The admin changes parameters live, starts an earthquake, and then plays the game log again.

6. M2 Network and Synchronization

Team: Tuğberk Akbulut, Ayşe Poyraz, Hasan Hüseyin Özdemir, Umutcan Ocak

6.1 Purpose

M2 connects the phones, the admin page and the LED board to the game server. It does not make game decisions. It takes messages from outside, checks them and gives them to M1. After each tick it takes the state from M1 and sends it to everybody. M2 also makes sure that nothing from outside can change the game state directly. For the admin, it measures the network and it can add delay and packet loss on purpose.

Runs on	Main computer, in the same server program as M1
Technology	TypeScript on Node.js; WebSocket for phones and admin; TCP and UDP for the LED board; MessagePack encoding; mDNS announcement; QR code for joining
Requirements	R1 R9 R10 R11

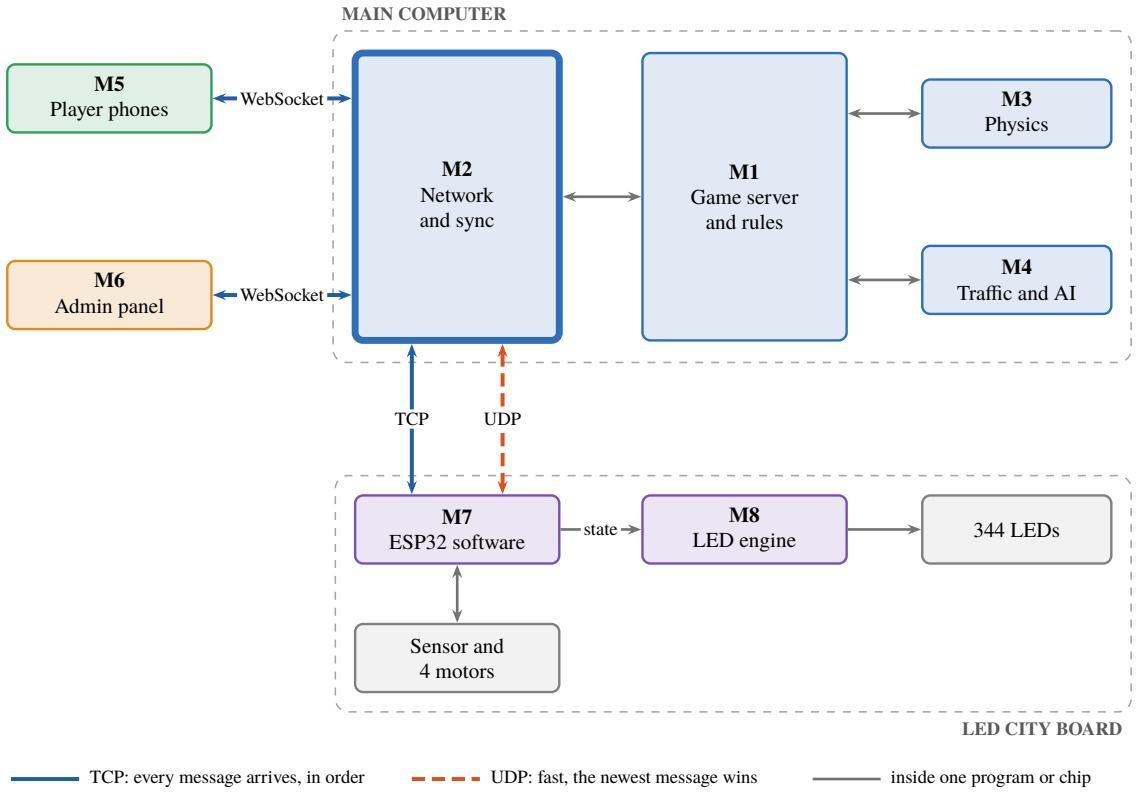


Figure 8: Where M2 sits in the system. All traffic from outside passes through M2.

6.2 Requirements covered

ID	Requirement	What M2 does for it
R1	Multiplayer game	Up to 8 phones join with a QR code or a 6-digit game code, only with the browser
R9	Live admin panel	Admin channel for commands, parameters, state, metrics and board data
R10	Network and measurement	Ping, packet loss and frame rate for each player; added delay and loss for each player and for the board
R11	Single authority	Inputs are checked and queued; every phone gets the same frozen state of each tick

6.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Phone messages	M5	Hello with player key, name and game code; join with team; controls; “Find me”; ping replies
In	Admin messages	M6	Login, commands, parameter changes, requests for parameters and the game log
In	Board messages	M7	Earthquake events, sensor samples, board status, delay replies
In	State and metrics	M1	Frozen game state and board state every tick; metrics twice per second
Out	To M1	M1	Checked controls, joins, “Find me”, admin commands, board events; network metrics

Dir.	Data	From / to	Content
Out	To phones	M5	Game state 30 times per second; join replies; ping; frame-rate limit
Out	To admin	M6	State 10 times per second, metrics, replies, board data
Out	To board	M7	Board state 10 to 30 times per second; config; vibrate commands

6.4 Main components

Component	What it does	Related modules
Web server and join	Serves the phone and admin pages, the QR code and the game code; announces the server on the local network	M5, M6, M7
Sessions	One session for each phone and the admin; player key for reconnecting within 30 s; one admin at a time	M1, M5, M6
Codec and checker	MessagePack encode and decode; checks type, fields and value ranges; a bad message gets an error reply	all
Input queue	Holds checked inputs until M1 reads them at the start of the next tick	M1
State broadcaster	Encodes the frozen state once and sends the same bytes to every phone	M1, M5, M6, M7
Delay and loss emulator	An up queue and a down queue for each player and for the board, set by the admin	M6
Ping and metrics	Ping twice per second, clock offset for the phone, ping, loss and frame-rate metrics	M1, M5, M6
Board link	Server side of the board connection: one TCP channel and one UDP stream	M7

6.5 How it works

Which link uses which protocol. This choice is important, so we made it message by message. A web browser can only open a WebSocket for two-way talk, so phones and the admin page use WebSocket. WebSocket runs on TCP, so every message arrives and the order is kept. The LED board is not a browser, so it can use both TCP and UDP. On TCP it gets the messages that must never be lost: the config (player colours, brightness, current limit) and the vibrate commands. The board also sends earthquake events and its status report on TCP. If one of these is lost, the game goes wrong, so TCP sends it again. The fast and repeated data goes on UDP: the board state, which comes 10 to 30 times per second, the sensor samples for the seismic graph, and the small delay probes. Here only the newest message is useful. If one UDP packet is lost, the next one comes 33 to 100 ms later and replaces it. TCP would wait for the lost packet and the LEDs would stop for a moment, so UDP is the better choice for this data. Each board state fits in one UDP packet (under 1 KB), so a packet is never split.

Joining. At the start the admin screen shows a QR code and a 6-digit game code. A player scans the QR code with the phone camera. The phone opens the game page from the main computer, so there is nothing to install. A player without a camera can open the short local address and type the code. The phone then chooses a team. The first time, the browser saves a random player key. If the page reloads or Wi-Fi drops, the phone sends the same key and gets the same motorcycle back within 30 seconds. The LED board finds the server by itself, because the server announces its name and port on the local network with mDNS.

Keeping the game state safe. Many phones send messages at the same time, but only M1 may change the game state, and only inside a tick. So M2 never writes to the state. A phone can only send controls; it can never send a position or a score. M2 decodes each message, checks its type and fields, and cuts every value into its range (for example steer is kept between -1 and 1). Each session can only control its own player, so one phone cannot move another motorcycle. Every message has an order number. A message with an old or repeated number is dropped. Good inputs wait in a queue, and M1 reads the queue at the start of the next tick. At the end of the tick M1 gives a frozen copy of the state. M2 encodes it once with MessagePack and sends the same bytes to all phones, so all players see exactly the same tick. The full game state is under 4 KB.

Ping, delay and loss. Twice per second M2 sends a ping with the server time. The phone answers with its own time, its frame rate and the number of the last state it got. From this M2 finds the round-trip time and the clock offset of the phone. Gaps in the order numbers give the packet loss. These values go to the admin graphs. For tests the admin can make the network worse for one player. Each player has an up queue and a down queue. A queue drops a message with the loss chance that the admin set, and holds the other messages for half of the added ping. The same tool works on the board link. The admin can also set a frame-rate limit, which M2 sends to the phone.

6.6 Testing and demonstration

M2 is tested with a fake M1 that sends a moving test state, with up to 8 scripted bot phones, and with the board emulator of M7. The tests check the message codec, bad and broken input, the load with 8 phones, how exact the added delay and loss are, reconnecting within 30 seconds, and the board link with 20% packet loss. In the demonstration phones join with the QR code or the game code. Then the admin changes one player's ping and loss live, and we watch the change on that phone and in the graphs.

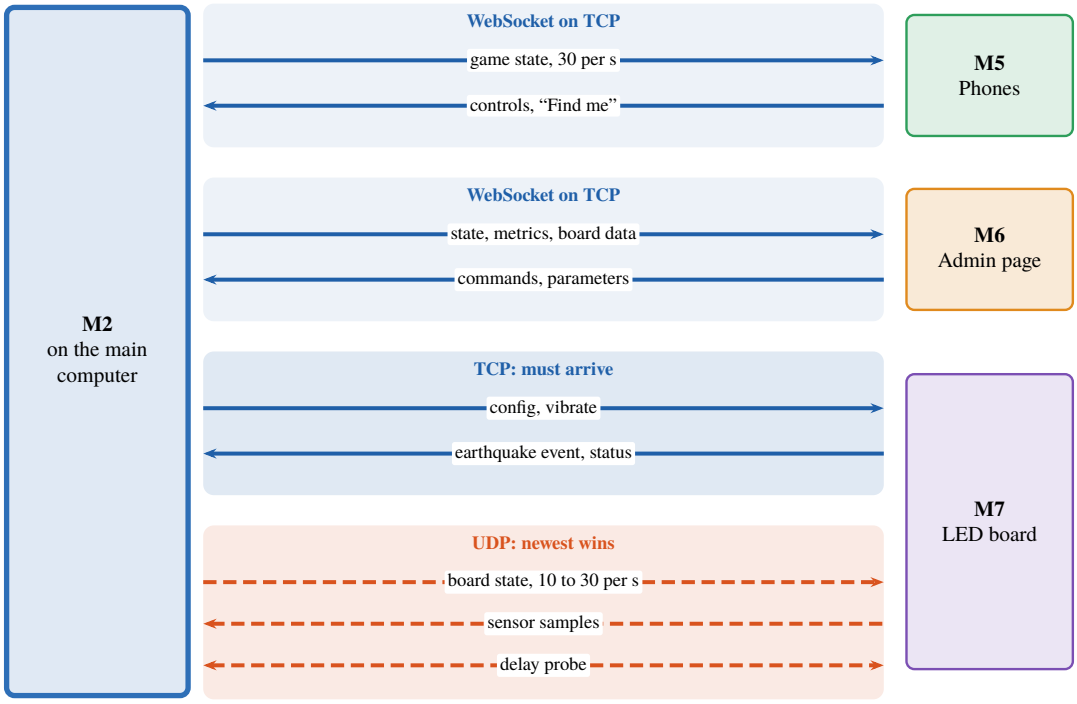


Figure 9: Every link and every message type. Blue solid arrows use TCP, orange dashed arrows use UDP.

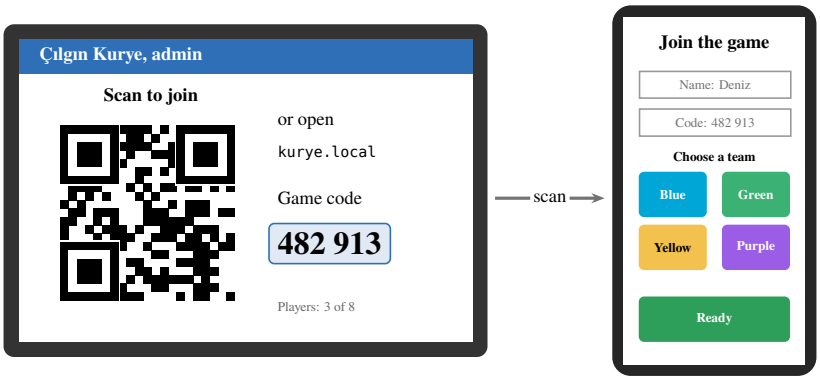


Figure 10: Joining: the QR code and game code on the admin screen, and the join page on a phone.

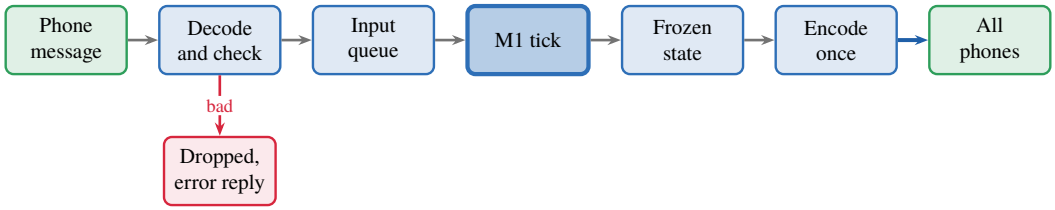


Figure 11: The path of an input. Only M1 writes the state, and everybody gets the same bytes.

7. M3 Physics and Collision

Team: Abdullatif Yılmaz, Hasan Alperen Elmas, Batuhan Yanar, Selim Barut

7.1 Purpose

M3 moves the motorcycles. It turns throttle, brake and steering into speed, direction and position. It finds collisions, decides when a motorcycle falls, and adds the effect of obstacles and earthquakes to the motion. The same code runs in two places: on the server, where its result is the real one, and on the phone, where it guesses the player's own motorcycle before the server answers.

Runs on	Main computer inside the game server; phone browser for prediction only
Technology	TypeScript library; our own simple 2D arcade physics, no ready physics engine; circle and box collisions on a uniform grid; seeded random numbers
Requirements	R3 R4 R6 R7 R11

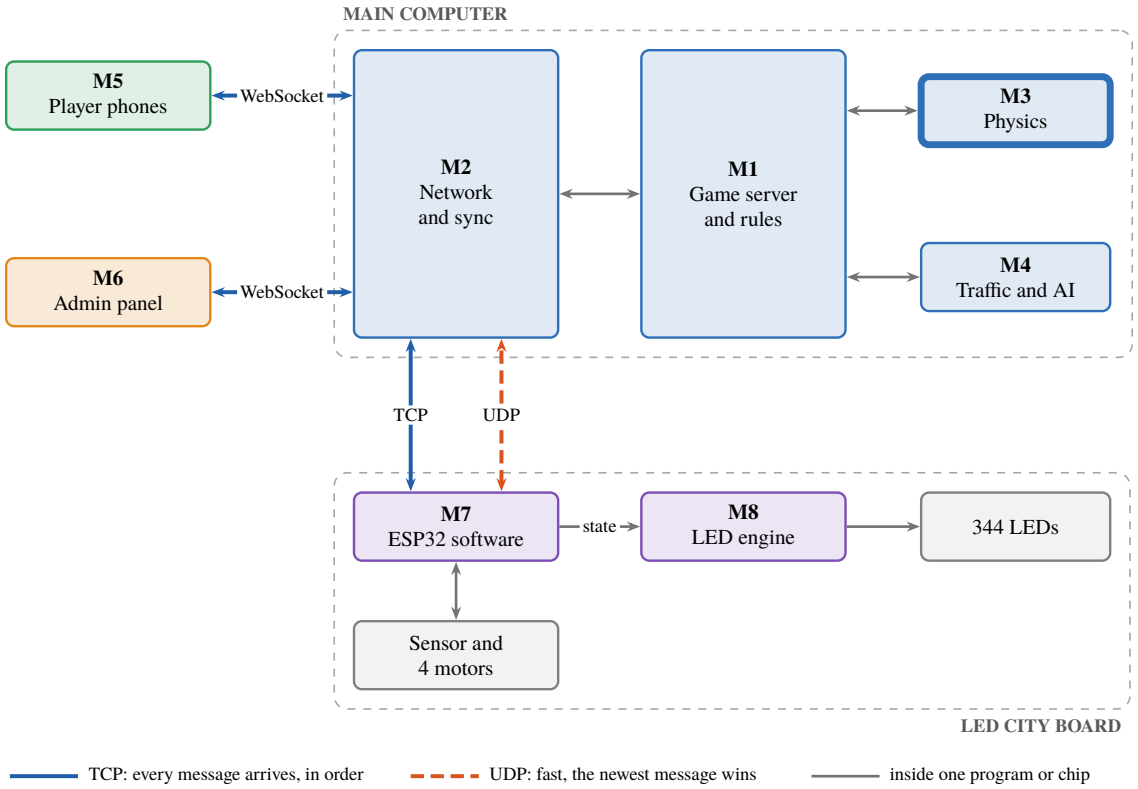


Figure 12: Where M3 sits in the system. M1 calls M3 in every tick; M5 uses a copy of M3 on the phone.

7.2 Requirements covered

ID	Requirement	What M3 does for it
R3	Driving and controls	Throttle, brake, left and right; no reverse; U-turn by steering in about a 3 m circle at low speed
R4	Collisions and falls	Collisions with buildings, barriers, NPC cars and motorcycles; a fast hit makes the motorcycle fall
R6	Obstacles	Road work and accidents are solid; an accident has a slow zone; a puddle makes the motorcycle slide
R7	Earthquake	Side push and less grip inside the earthquake radius that M1 sends
R11	Single authority	The code gives the same result for the same input; the server result always wins; the phone only predicts

7.3 Inputs and outputs

Dir.	Data	From / to	Content
In	City plan	Shared file	Roads, lanes, blocks and buildings as fixed colliders
In	Controls	M1	Every tick: throttle, brake, steer of humans and bots; a speed factor for each player
In	Obstacles, NPC cars	M1, cars from M4	Obstacle type, road, lane, position; boxes of the NPC cars
In	Earthquake	M1	Centre, strength, radius; the players M1 decided to drop
In	Parameters	M6, through M1	Physics group: top speed, grip, fall speeds, puddle and earthquake effects

Dir.	Data	From / to	Content
Out	Poses	M1	Position, direction, speed, lean, sliding, fallen
Out	Events	M1 and M4	Collisions (with what, how fast) and falls
Out	Predicted pose	M5	The player's own motorcycle before the server answers

7.4 Main components

Component	What it does	Related modules
Motorcycle model	Speed, steering, turning, grip and sliding	—
Colliders	Buildings and the outer wall from the city plan; obstacles as boxes, slow zones and slippery areas	M1
Collision solver and fall judge	Solves contacts in a fixed order and decides falls from the hit speed	M1, M4
Earthquake effect	Push and less grip inside the radius	M1
Predictor	Prediction and correction on the phone	M5

7.5 How it works

One physics step. The server runs 30 ticks per second, and M3 runs 2 sub-steps in each tick, so one sub-step is about 16 ms. Small steps keep fast motorcycles from passing through thin walls. Each sub-step does five things in a fixed order, as the figure shows. A fixed order and seeded random numbers mean that the same inputs always give the same result on the server and on the phone.

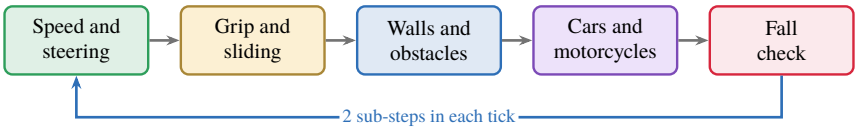


Figure 13: The five parts of one physics sub-step.

Driving. The throttle adds speed up to the top speed, and the brake and air drag take speed away. There is no reverse. The steering changes the direction. At low speed the motorcycle can turn in a circle of about 3 metres, so a U-turn is easy. At high speed the turn is wider. The model also gives a lean angle, which M5 uses to tilt the 3D motorcycle in a turn.

Grip and sliding. In a turn the motorcycle needs a side force. M3 compares the size of this force with the grip limit. If the force is below the limit, the motorcycle follows the turn. If it is above the limit, the motorcycle slides out of the turn. We compare only the size, so a left turn and a right turn act the same. On a water puddle the grip limit is very low, so the motorcycle slides but does not slow down. Inside an earthquake radius the grip is lower too, and there is a small push to the side.

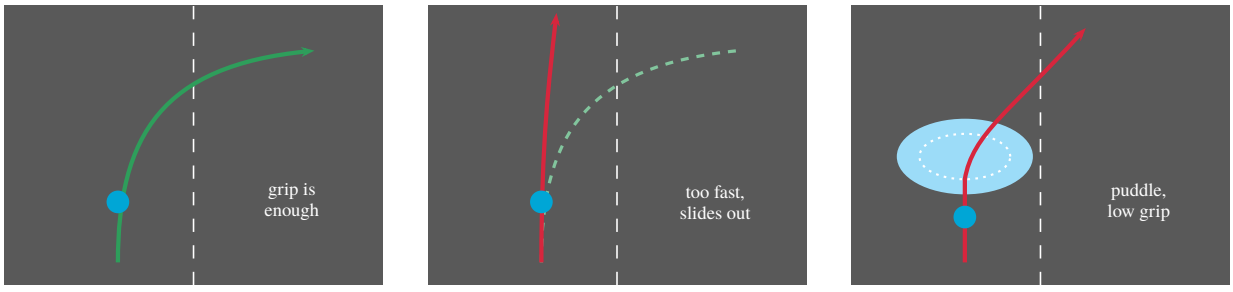


Figure 14: Three cases of grip, seen from above. Green is the wanted path; red is the real path.

Collisions and falls. Buildings, the outer wall and the road-work barriers come from the city plan and the obstacle list as boxes. NPC cars are boxes, and motorcycles are circles. To find collisions fast, M3 puts every object into a uniform grid, so it checks only objects in the same or next cell. A contact pushes the objects apart. If the hit speed is above the fall speed that the admin set, the motorcycle falls. When two motorcycles hit each other and their relative speed is above the limit, both of them fall. A fallen motorcycle is not solid, so it does not block the road for others. M1 gives the penalty time and later asks M3 to stand the motorcycle up again. An accident is solid in one lane and has a slow zone around it.

Prediction on the phone. On the phone, the player must see the own motorcycle move at once, not 100 ms later. So M5 runs a copy of M3 for this one motorcycle. Each control gets a number and is kept in a list. When a server state arrives, the phone takes the real position from the server, throws away the controls that the server has already used, and runs the remaining controls again. If the new result is close to what the player sees, the difference is blended away in a short time. If it is far, for example after a crash, the motorcycle jumps to the right place.

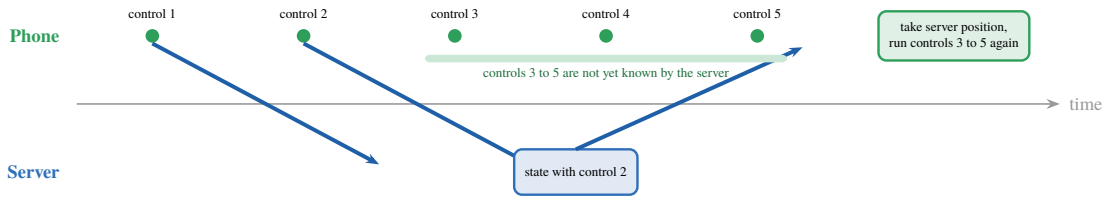


Figure 15: Prediction and correction for the player's own motorcycle.

7.6 Testing and demonstration

M3 needs no other module. A test makes a world from the city plan and gives scripted controls. We test a straight run and braking, a U-turn, slides to the left and to the right, wall and car crashes, the earthquake radius, the same result for the same inputs, and prediction with delay and loss. In the demonstration one motorcycle drives in the city with the keyboard. The admin changes grip and top speed, and we see the motorcycle change at once.

8. M4 Traffic and AI

Team: Hasan Alperen Elmas, Batuhan Yanar, Muhib Taha Boy

8.1 Purpose

M4 brings the city to life. It finds routes and travel times on the road network. It moves the NPC cars that make the traffic. It also drives the AI bots that fill the empty places in a game. A bot makes the same control message that a phone sends, so for M1 a bot and a human player look the same.

Runs on	Main computer inside the game server; M1 runs it once in every tick
Technology	TypeScript; our own shortest-path search (Dijkstra); a car-following model for NPC cars; a state machine and path following for bots; seeded random numbers
Requirements	R1 R2 R4 R6 R11

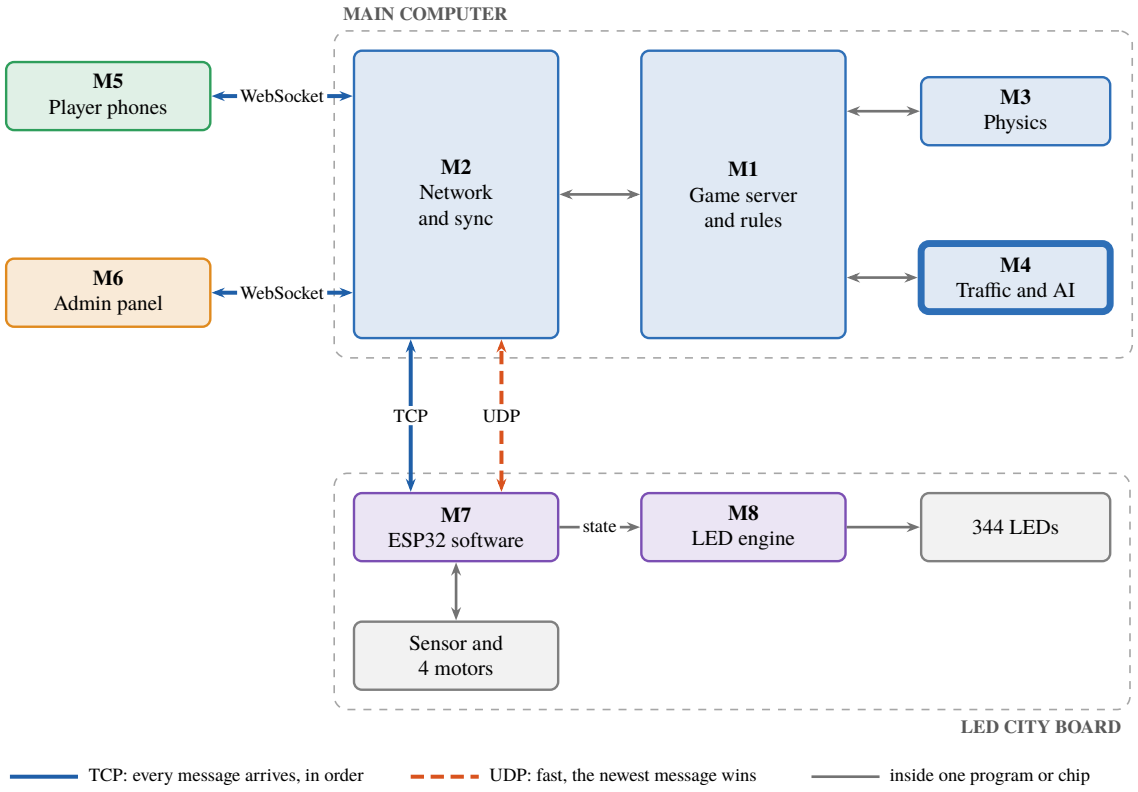


Figure 16: Where M4 sits in the system. M4 talks only to M1, inside the same program.

8.2 Requirements covered

ID	Requirement	What M4 does for it
R1	Multiplayer game	Bots take the empty places and play like players
R2	Delivery cycle	Travel-time table from each store, which M1 uses for fair near and far points
R4	Traffic	NPC cars make real traffic on the roads
R6	Obstacles	Heavy traffic adds slow cars; an accident closes a lane; road work closes the road for routes and cars
R11	Independent development	Talks only to M1 with plain data; tested with its own test program

8.3 Inputs and outputs

Dir.	Data	From / to	Content
In	City plan	Shared file	Intersections, roads, lanes, stores, possible delivery points
In	Game data	M1	Every tick: players, obstacles with a version number, car-hit events
In	Parameters	M6, through M1	Number of NPC cars, traffic effects, bot skill
Out	NPC cars	M1, then M3, M5, M6	Position, direction, speed, lane, size
Out	Bot controls	M1, then M3	The same record as a phone: throttle, brake, steer, deliver
Out	Travel times	M1	Seconds from each team store to every possible delivery point
Out	Traffic metrics	M1, then M6	Number of NPC cars, mean NPC speed, bot route changes

8.4 Main components

Component	What it does	Related modules
Road graph	36 intersections and 60 two-way roads with lanes and turn costs	—
Route planner	Fastest routes and the travel-time table; computed again only when the obstacles change	M1
NPC traffic	Moves, adds and removes NPC cars	M1, M3
Bot drivers	One state machine for each bot	M1, M3
Traffic metrics	Values for the admin graphs	M6

8.5 How it works

The road graph and routes. The city has 6 roads in each direction. Where two roads cross, there is an intersection, so the graph has 36 intersections. Between them there are 60 road segments, and cars can drive both ways. Each segment has a cost: the expected time to drive it. Turning left or making a U-turn adds a small extra cost. Obstacles change the costs. Heavy traffic and an accident make a segment slower, and road work closes it. M4 uses Dijkstra's shortest-path search to find the fastest routes. The graph is small, so one search takes much less than a millisecond. M4 computes the routes again only when the version number of the obstacle list changes. From the results it makes the travel-time table: the time from each of the four stores to every possible delivery point. M1 uses this table to choose fair near and far points.

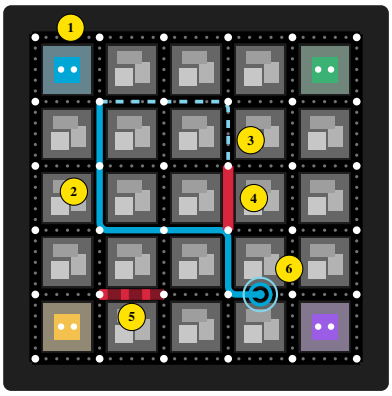


Figure 17: A route on the road graph, drawn on the board. Obstacles change the cost of the roads.

NPC traffic. Each NPC car follows a lane and picks a new turn at each intersection. A car keeps a safe gap behind the car or the motorcycle in front of it. If the gap gets small, it slows down smoothly, and it speeds up again when the road is free. A car enters an intersection only when the intersection is free, so cars do not drive through each other. New cars never appear close to a player. If a car is stuck for 10 seconds, M4 removes it and adds it again somewhere else, so the traffic never locks up. The admin sets the number of cars, and heavy-traffic obstacles add extra slow cars on their road.

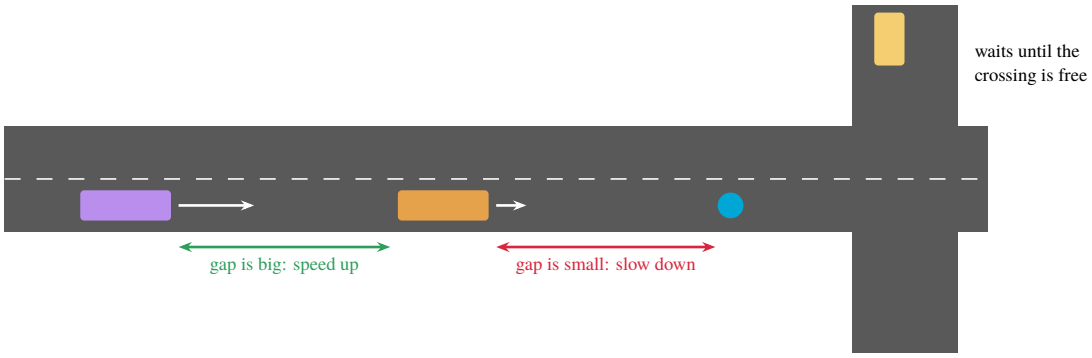


Figure 18: How NPC cars drive: a safe gap behind the next vehicle, and a free crossing before they enter.

Bots. Every bot has a small state machine that follows the delivery cycle, as the figure below shows. The bot drives to the store, gets the package, drives to the delivery point, slows down and presses "Deliver", and then goes back. On the way it follows the route from the planner. It steers toward a point a little ahead on the path, so the motion looks smooth. If it falls, it waits for the penalty time. If it is stuck, it asks for a new route. One "bot skill" value sets the top speed, the reaction time and some steering noise, so the admin can make bots weaker or stronger.

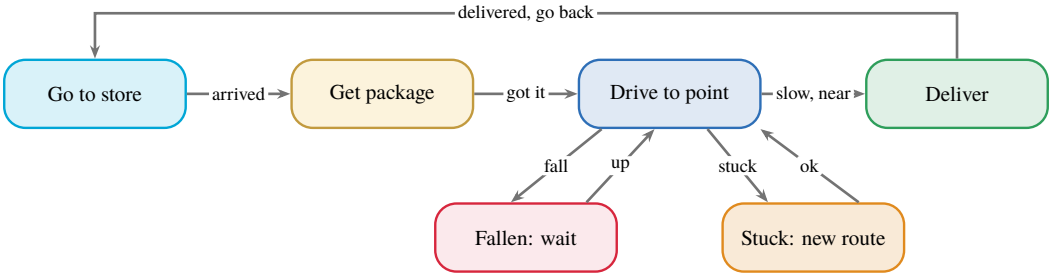


Figure 19: The state machine of one bot.

8.6 Testing and demonstration

A small test program plays the role of M1 and draws the city from above. We test routes with and without closed roads, equal travel times for all teams in a city that is the same on all sides, 80 cars for 30 minutes without a traffic lock, obstacles that change during play, 8 bots that finish deliveries, and the same result with the same seed. In the demonstration the admin adds obstacles and more traffic, and the cars and bots change their way at once.

9. M5 Player Client and 3D City

Team: Abdullatif Yılmaz, Berk Andaçgil, Selim Barut

9.1 Purpose

M5 is the web page on each player's phone. The player scans a QR code or types the game code, chooses a team, and drives a motorcycle in a low-poly 3D city with buttons on the screen. There is no map on the phone. Only a compass arrow shows the direction of the delivery point or of the store. So players must look at the LED board on the table to find their way.

Runs on	Player phone, only the web browser, nothing to install; the page comes from M2
Technology	TypeScript web page built with Vite; Three.js for 3D; low-poly glTF models from one Blender file, shared with the board buildings; MessagePack; the M3 physics code for prediction
Requirements	R1 R2 R3 R5 R7 R8 R10 R11

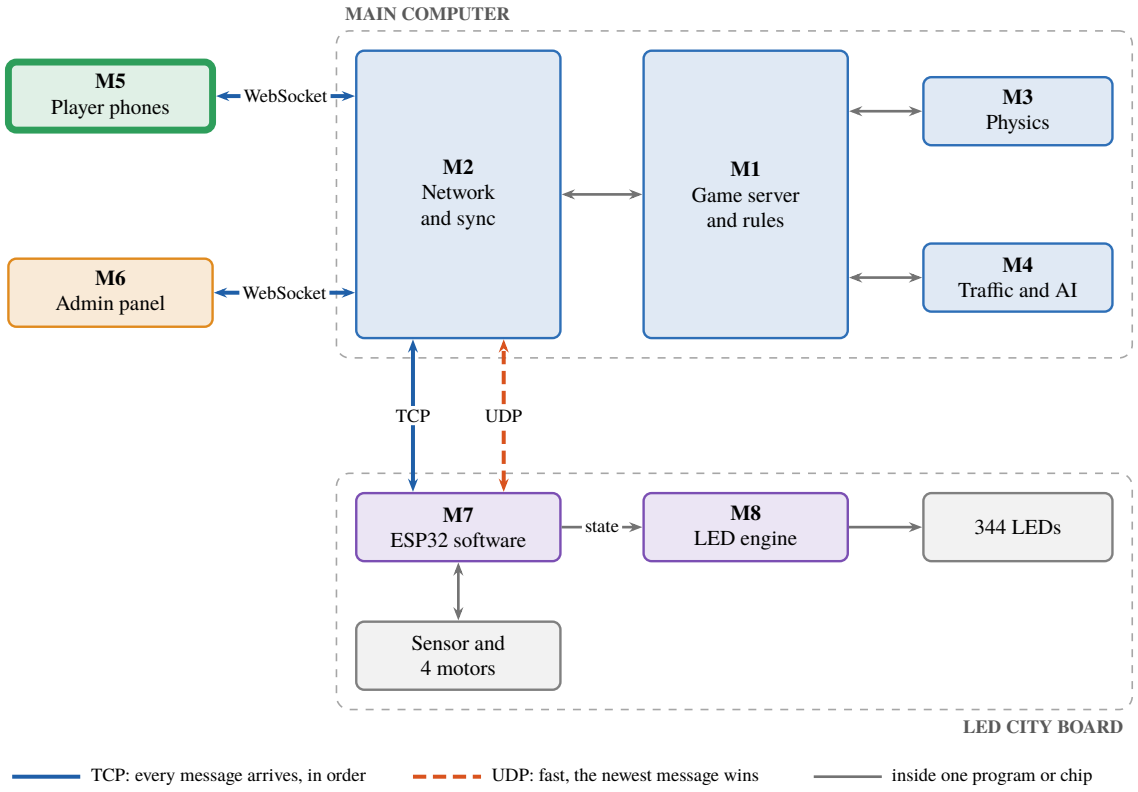


Figure 20: Where M5 sits in the system. Each phone talks to M2 over one WebSocket.

9.2 Requirements covered

ID	Requirement	What M5 does for it
R1, R2	Game and deliveries	Join, team choice, timer, scores, package icon, "Deliver" button
R3	Driving and controls	Gas, brake, left and right buttons
R5	No map on the phone	Only a compass arrow; "Find me" button
R7	Earthquake	The camera shakes
R8	Same city as the board	The 3D city is built from the shared city plan and the same models
R10	Network and measurement	Frame-rate limit from the admin; frame-rate report
R11	Single authority	Own motorcycle is predicted at once and corrected by the server

9.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Game state	M2	30 times per second: players, NPC cars, obstacles, scores, compass target, "Find me" waiting time
In	Join reply, ping, net config	M2	Player, team, colour tone; server time; frame-rate limit
In	Touch	Player	Buttons
Out	Controls	M2, then M1	On every change and at least 10 times per second: throttle, brake, steer, deliver

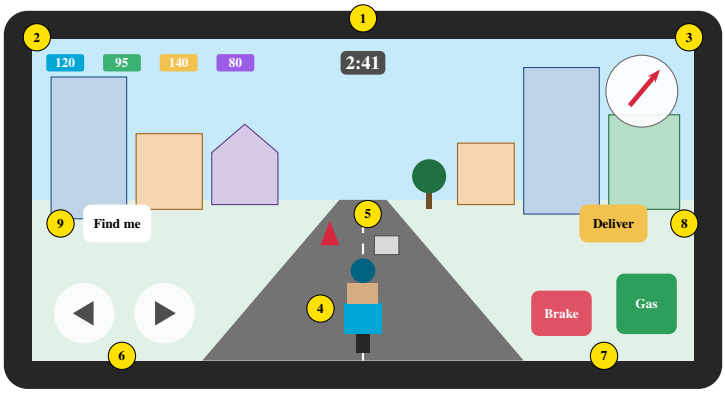
Dir.	Data	From / to	Content
Out	Join, “Find me”	M2, then M1	Player key, name, game code, team; “Find me” request
Out	Ping reply	M2	Phone time, frame rate, last state received

9.4 Main components

Component	What it does	Related modules
Screens and HUD	Join, lobby, countdown, game and end screens; timer, scores, compass	M1
Controls	Turns button presses into the control record	M1, M2
Network client	WebSocket link, MessagePack, reconnect with the saved player key	M2
Prediction	Own motorcycle with the M3 code; correction on each state	M3
3D city and scene	City from the city plan; motorcycles, cars, obstacles, markers; camera	M4, M8

9.5 How it works

The game screen. The phone is held sideways. The 3D view fills the screen, and the camera follows the motorcycle from behind. At the top the player sees the time left and the score of each team. In the top corner there is the compass. The arrow points to the delivery point, or to the store when the player has no package. It never shows the distance or the roads. The buttons are large so that the thumbs can find them without looking: left and right on one side, gas and brake on the other side. The “Deliver” button is always active. If the server says no, a short message shows the reason, for example “too fast”. The “Find me” button asks the server to flash the player’s LED on the board in white for 1 second. After a press, the button waits for the time that the server sends.



1 Time left
2 Team scores
3 Compass arrow, no map
4 Own motorcycle, team colour
5 NPC car and an obstacle
6 Left and right
7 Gas and brake
8 Deliver button
9 “Find me” button

Figure 21: Planned game screen on a phone held sideways.

Screens. The page has five screens. On the join screen the player types a name and, if there was no QR code, the game code. In the lobby the player chooses one of the four teams and waits. A 3 second countdown starts the game. The game screen is shown above. At the end, the page shows the winner and the scores of all teams.

Smooth play on a real network. The server sends a state 30 times per second, but Wi-Fi adds delay. If the phone waited for the server, the motorcycle would react late to a button. So the phone moves its own motorcycle at once with the M3 code, and corrects it when the real state arrives. For the other players and the NPC cars, the phone keeps the last few states and draws them 100 ms in the past. Then it can always move them smoothly between two real states, and they do not jump.

The 3D city. The city is built from the same city plan as the LED board: 6 roads in each direction, 60 road segments and 25 blocks. Tall buildings stand in the centre, houses and trees are outside, and the four team stores in the corners have the team colours. The models are low-poly and come from one Blender file. The board uses the same models for 3D printing (1 m in the game is 1 mm on the board), so a player can find the same building on the board and on the phone. Objects that repeat, like trees and houses, are drawn as instances, so a mid-range phone can draw at least 30 frames per second. During an earthquake the camera shakes, and the shaking is stronger near the centre of the earthquake.

9.6 Testing and demonstration

Tests run against a fake server. We test joining on Android and iPhone, every button, prediction with 150 ms delay and 10% loss, at least 30 frames per second on a mid-range phone, and reconnecting after a page reload. In the demonstration two phones drive in the 3D city. The players use the compass and the board to find the delivery point, and the admin changes the frame-rate limit of one phone.



Figure 22: The five screens of the phone page.

10. M6 Admin Panel and Monitoring

Team: Batuhan Yanar, Ayşe Poyraz, Berk Andaçgil

10.1 Purpose

M6 is the admin screen. On one page the admin starts and controls the game, changes all parameters during play, puts obstacles on a live map, starts earthquakes and watches the results as live graphs. Each change can be seen in three places: in the 3D city on the phones, on the LED board, and in the graphs on this page.

Runs on	Browser on the main computer, or a laptop on the game Wi-Fi; the page comes from M2
Technology	TypeScript web page built with Vite; uPlot for live graphs; HTML canvas for the live map; MessagePack
Requirements	R6 R7 R9 R10 R11

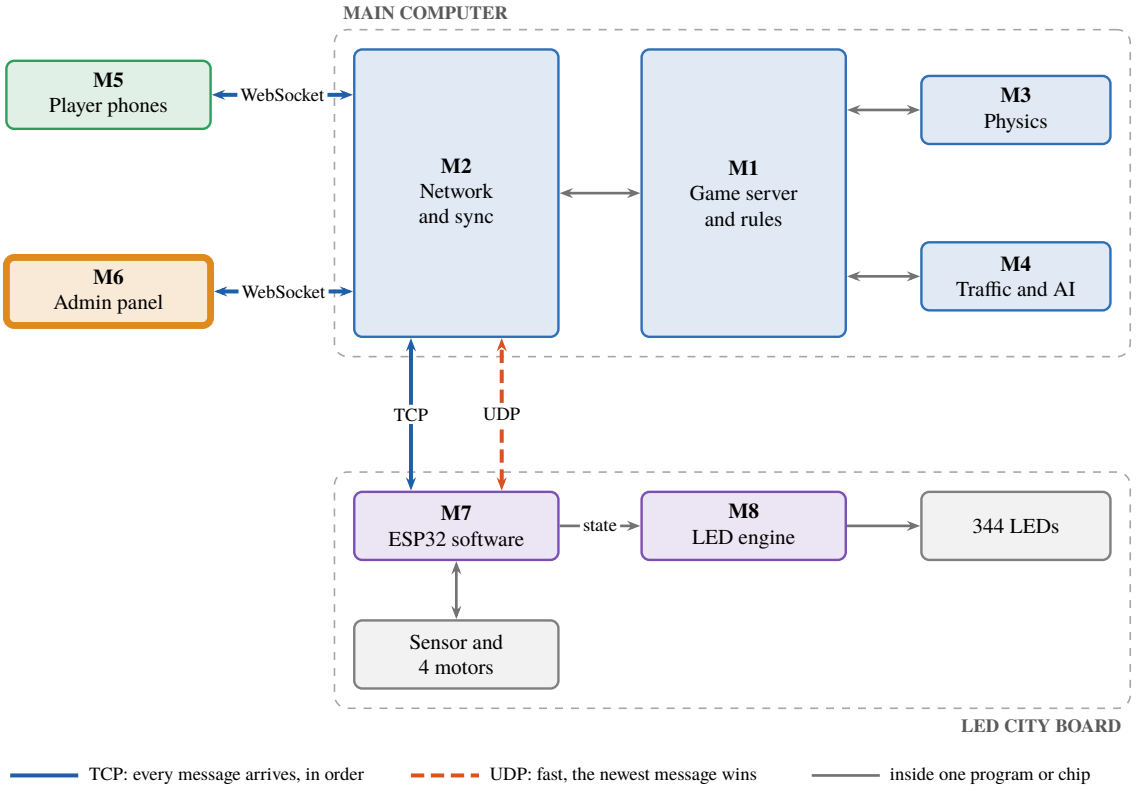


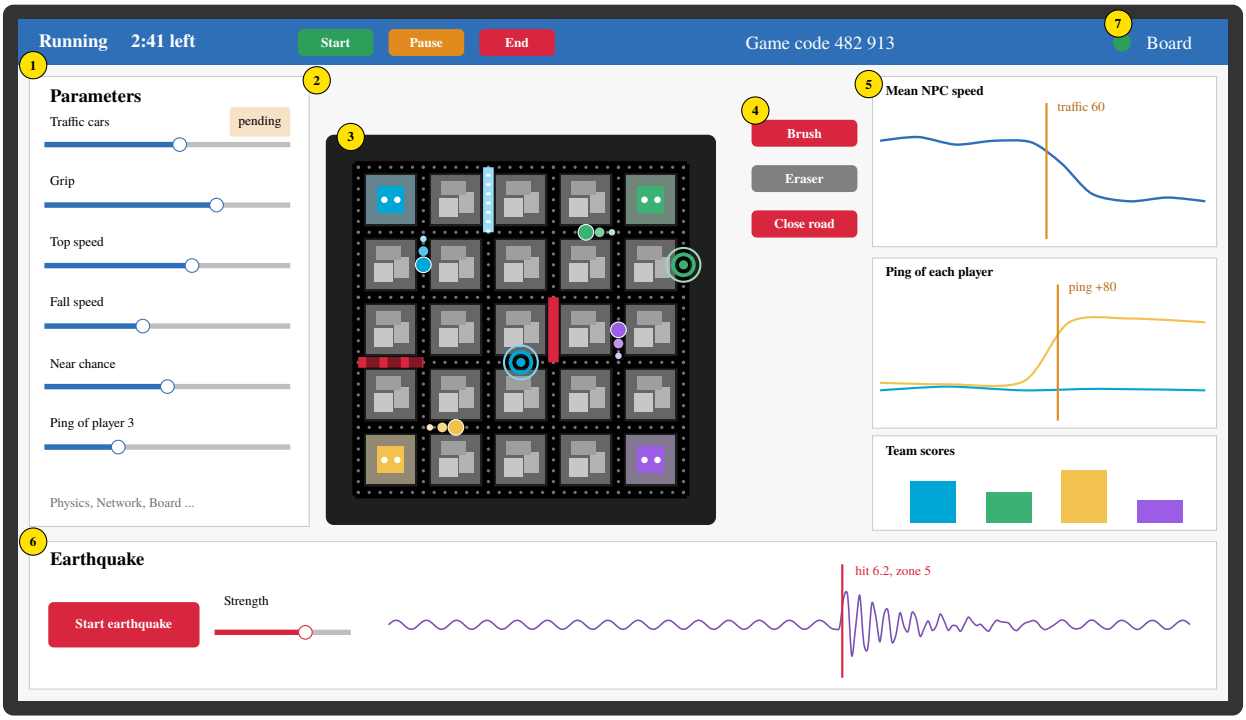
Figure 23: Where M6 sits in the system. The admin page talks to M2 over one WebSocket.

10.2 Requirements covered

ID	Requirement	What M6 does for it
R6	Obstacles	Map tools: obstacle brush, eraser, close a road, obstacle in front of a player, traffic density
R7	Earthquake	Earthquake panel with strength and a live seismic graph
R9	Live admin panel	Parameter forms for every group; graphs with a mark for every change
R10	Network and measurement	Ping, loss and frame-rate graphs; network and board settings
R11	Independent development	Works against a fake server with recorded data

10.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Game state	M1, through M2	10 times per second, for the live map
In	Metrics	M1, through M2	Twice per second: scores, delivery times and distances, speeds, falls, traffic, ping, loss, frame rate, board delay
In	Parameters, replies	M1, through M2	Every parameter with its value and limits; ok or error for each command
In	Board data	M7, through M2	Sensor samples (100 per second), board status, earthquake events
Out	Commands	M2, then M1	Parameter changes; start, pause, end, reset; obstacles; earthquake; new game code
Out	Files	Admin	Graph data as CSV; game log



1 Game phase, time and buttons
2 Parameter forms
3 Live map of the city
4 Map tools
5 Live graphs with change marks
6 Earthquake panel and seismic graph
7 Board link light

Figure 24: Planned layout of the admin page.

10.4 Main components

Component	What it does	Related modules
Top bar	Phase, time left, game buttons, QR code and game code, board status light	M1, M2
Parameter forms	One part for each group; “pending” until the reply; goes back to the old value on an error	M1–M4
Live map	City from above with players, cars, obstacles, delivery points; map tools	M1, M4
Graphs	Live graphs with change marks; CSV export	M1, M2, M7
Earthquake panel and log	Start button, seismic graph; replies, events, errors	M1, M7

10.5 How it works

The page. The figure below shows the planned layout. The top bar shows the game phase, the time left, the buttons to start, pause and end the game, the game code and a small light for the board link (green when the board is connected). On the left are the parameter forms. In the centre is the live map: the city from above, drawn on a canvas 10 times per second with all motorcycles, NPC cars, obstacles and delivery points. On the right are the live graphs. At the bottom are the earthquake panel and the seismic graph.

Parameters. The parameters are in groups: game, obstacles, players, physics, earthquake, network (one set for each player) and board link. When the page opens, it asks M1 for all parameters with their limits and builds the forms from this list, so a new parameter needs no new page code. A slider sends its value when it stops moving for 300 ms, so one drag does not send 50 messages. Until the server answers, the value is marked “pending”. If the answer is “ok”, the mark goes away. If the answer is an error, the slider goes back to the old value and the log shows the reason.

Cause and effect. The main goal of the panel is to show the effect of a change. So every change puts a thin vertical line on all graphs, with the name of the parameter. For example, when the admin sets the traffic to 60 cars, the line appears and the graph of the mean NPC speed goes down right after it. At the same time the phones show more cars and the board shows more red. The graph data can be saved as a CSV file for the report.

Map tools. The admin can paint obstacles on the live map. A click is turned into a road segment, a lane and a position with the numbering of the city plan, and this goes to M1 as a command. There is a brush for each obstacle type, an eraser, a tool to close a whole road, a button to put an obstacle just in front of one player, and a traffic density control.

Earthquake. The start button sends a command to M1, and M1 asks the board to run its motors. The seismic graph shows the sensor signal of the board live, 100 samples per second. Each earthquake event puts a mark on the graph with its strength and zone. A hand hit on the board shows on the same graph, so the admin can compare the two.

10.6 Testing and demonstration

Tests run against a fake server that plays back a recorded game. We test every parameter form, every map tool, all graphs and the CSV export, a 5-minute load test, and that a second admin login is refused. In the demonstration the admin changes parameters, places obstacles and starts an earthquake, and the audience sees each change on the phones, on the board and in the graphs.

11. M7 ESP32 Software and Communication

Team: Tuğberk Akbulut, Hasan Alperen Elmas, Hasan Hüseyin Özdemir, Muhib Taha Boy

11.1 Purpose

M7 is the software of the ESP32 inside the LED board, and the link between the board and the main computer. It gets the game state over Wi-Fi and gives it to the LED engine (M8). It reads the motion sensor and finds the strength and the place of a hit on the board. It drives the vibration motors. It also reports the health of the board and the sensor signal to the admin.

Runs on	ESP32 in the LED board: core 0 (the LED engine M8 runs on core 1)
Technology	C++ with the Arduino framework for ESP32 and FreeRTOS tasks; TCP and UDP over Wi-Fi with MessagePack; 6-axis motion sensor (accelerometer and gyroscope); 4 vibration motors with transistor drivers
Requirements	R7 R8 R10 R11

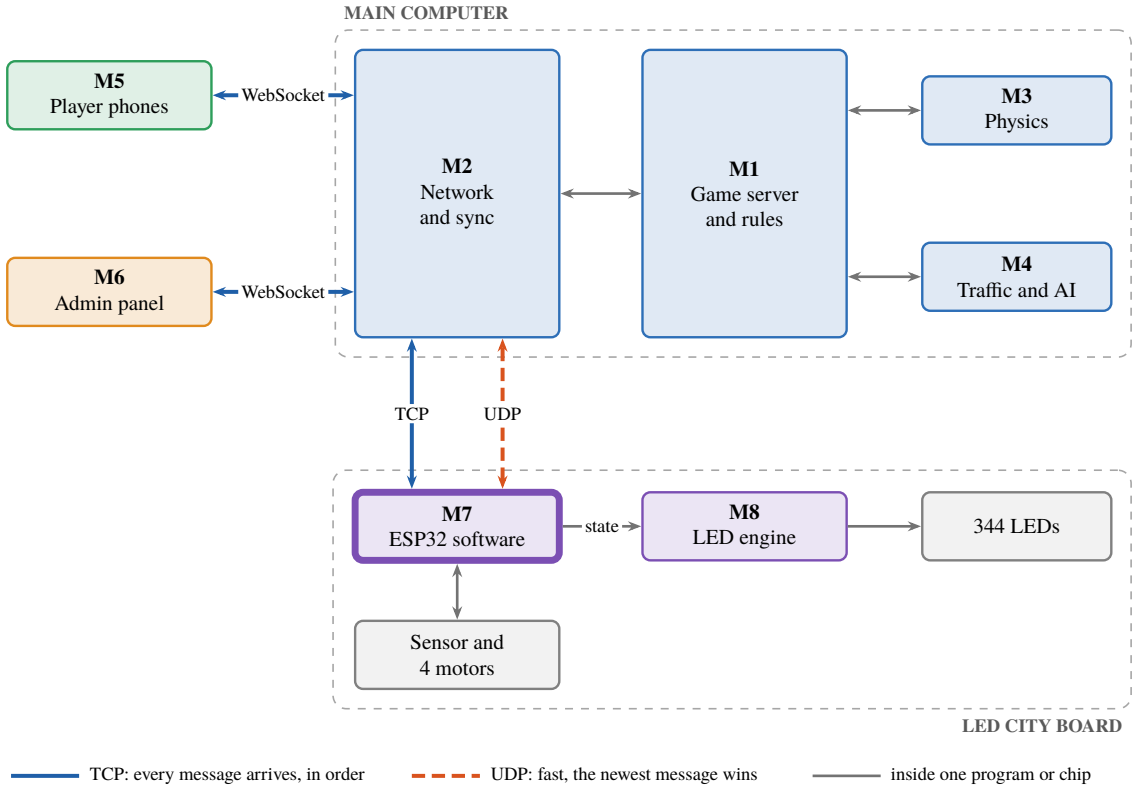


Figure 25: Where M7 sits in the system. M7 talks to the server on two channels: TCP and UDP.

11.2 Requirements covered

ID	Requirement	What M7 does for it
R7	Earthquake	Finds hand hits and motor shaking with the sensor; strength 0–10 and a 3 × 3 zone; drives the motors for the admin button
R8	Physical LED board	Brings the game state to the board over Wi-Fi; one ESP32 for the LEDs, the sensor and the motors
R10	Network and measurement	Board delay measurement; sensor samples for the seismic graph; comes back after a lost link
R11	Independent development	PC emulators for both sides of the link

11.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Board state	M2, UDP	10 to 30 per second: phase, players (team, tone, position, flags), delivery points, obstacles, earthquake
In	Config	M2, TCP	Player colours, brightness 0.1–1, LED current limit
In	Vibrate	M2, TCP	Motors to use, strength 0–1, duration of at most 5 s
In	Sensor	Hardware	1000 samples per second
Out	Newest state	M8	Latest board state, config and link state, on the same chip
Out	Earthquake event	M2, TCP, to M1 and M6	Source (hand or motors), strength 0–10, centre, zone 0–8, confidence

Dir.	Data	From / to	Content
Out	Sensor samples	M2, UDP, to M6	100 per second for the seismic graph, sent in groups of 10
Out	Status	M2, TCP, to M6	Once per second: frame rate, Wi-Fi signal, LED current, errors

11.4 Main components

Component	What it does	Related modules
Network link	Joins Wi-Fi, finds the server, keeps the TCP channel and the UDP stream, connects again after a loss	M2
Message codec	MessagePack encode and decode; checks the version and the order numbers; drops old or broken messages	M2
State hand-over	Two-slot buffer: the LED engine always takes the newest full state and never waits	M8
Hit detector	Finds hits in the sensor signal; computes the strength and the zone	M1, M6
Motor controller	Runs vibrate commands with a soft start and safety limits	M1, M6
Status and settings	Status reports and delay replies; Wi-Fi and calibration settings in flash, changed over USB	M6

11.5 How it works

Two channels. The board keeps one TCP channel and one UDP stream to the server. Messages that must arrive go on TCP: the config, the vibrate commands, the earthquake events and the status report. Data that comes again and again goes on UDP: the board state, the sensor samples and the delay probes. If one UDP packet is lost, the next one replaces it, so the LEDs never wait. Every message has an order number, so the board can drop a UDP packet that comes late and out of order.

Two cores. The ESP32 has two cores. Core 0 runs all M7 work: Wi-Fi, the network tasks, the sensor, the hit detector and the motors. Core 1 runs only the LED engine of M8, 60 frames per second. So Wi-Fi work never breaks the LED timing. The two sides share the state through a two-slot buffer. M7 always writes into the free slot. When the slot is full, M7 switches one index in one step. The LED engine reads the slot that the index shows, so it always gets a complete state and it never waits for a lock.

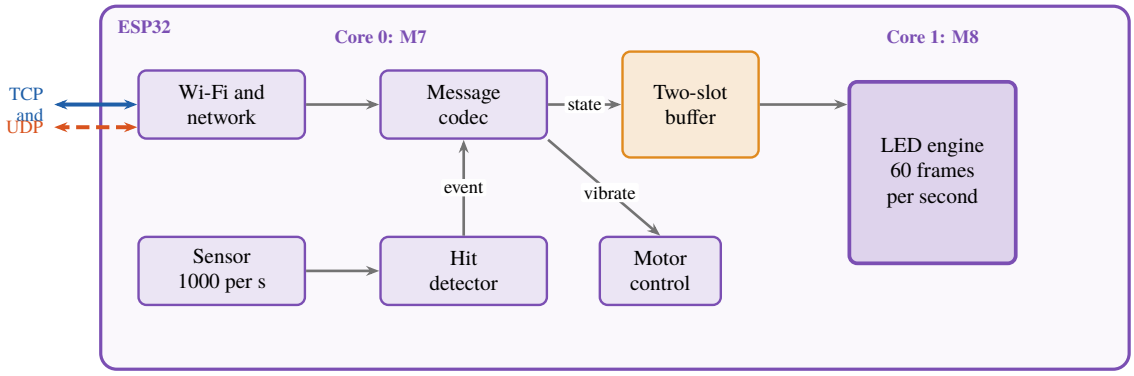


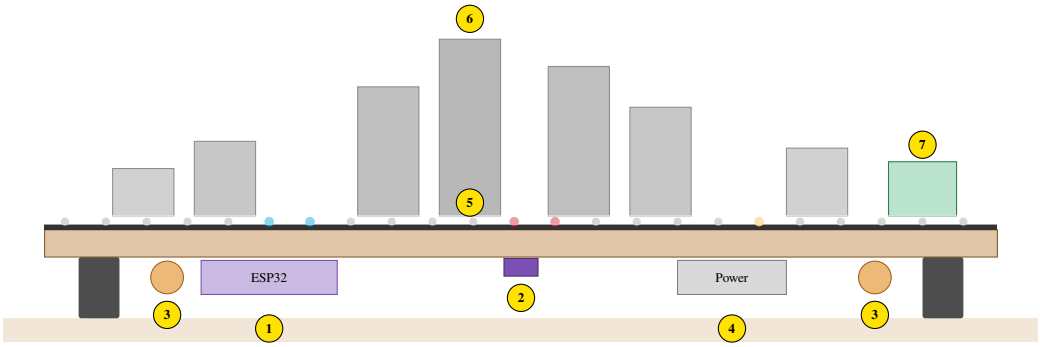
Figure 26: Work on the two cores of the ESP32. The buffer is the only shared part.

What is inside the board. The figure below is a side cut of the board. The motion sensor is fixed at the centre, under the plywood, so it feels a hit from every side. The four vibration motors are under the four corners. They have their own power line, so they do not make the LED voltage drop. The ESP32, the level shifter and the power supply sit under the board too.

Finding a hit. The sensor measures the motion 1000 times per second. First a filter takes away gravity and slow tilting, so only the shaking is left. The detector also learns the normal noise of the room. The start threshold is the mean noise plus six times its spread, so it goes up on a noisy table. When the signal passes the threshold, a hit starts. The detector takes a 150 ms window from this moment. The energy E in the window gives the strength with a log scale, like a real earthquake scale: strength = k × log10(E / E0). Here E0 is the energy of the weakest hit we accept, and k is set in calibration with a standard hit. The result is cut to 0–10. The first push of the board shows the direction of the hit, and its size shows how far the hit is from the centre. From this the detector picks one of 9 zones in a 3 × 3 grid and gives a confidence value. After a hit the detector waits 1 second, so one hit makes only one event. The event goes to the server within 200 ms.

Motors. When the admin presses the earthquake button, M1 sends a vibrate command on TCP. The command says which motors to use, the strength and the time, at most 5 seconds. M7 drives the motors with PWM through transistors. It starts slowly over 100 ms, so the current does not jump. The shaking is then measured by the same detector as a hand hit, and the event is marked with “motors” as its source. So the earthquake in the game follows the real shaking of the table.

When the link is lost. If no state comes for 2 seconds, M7 stops the motors at once and tells M8 to show a “searching” animation. Then it tries to connect again. The goal is to be back within 5 seconds after the router is up. The Wi-Fi name,



1 ESP32 and level shifter
2 Motion sensor at the centre
3 Vibration motors at the corners
4 5 V power supply
5 LED strips on the roads
6 3D-printed buildings
7 Team store with two LEDs

Figure 27: Side cut of the board with the parts that M7 uses.

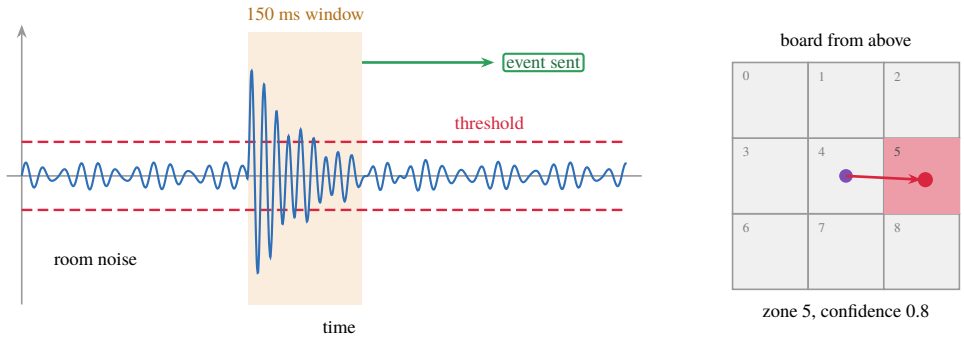


Figure 28: Left: the sensor signal of one hand hit. Right: the 3 × 3 zones; the purple dot is the sensor.

the password and the calibration values are stored in the ESP32 flash. We can change them over the USB cable, so we do not need to build the code again at a new place.

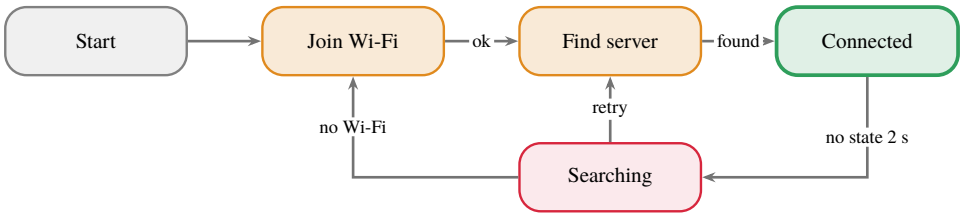


Figure 29: Link states of the board. In “Searching” the motors stop and the LEDs show a waiting animation.

11.6 Testing and demonstration

Two PC programs take the place of the neighbours. The server emulator sends test scenarios with delay and loss that we can change. It also shows the earthquake events and the seismic signal. The board emulator draws the board on the screen, so M1 and M2 can test before the real board is ready. The tests use the shared example messages, broken packets, a 2-hour run, added delay, 20% loss and a router restart. For the sensor we test hit finding and false alarms. Our targets are: at least 95% of calibrated hits are found; at most one false earthquake in 30 minutes of normal play; the correct quarter of the board in at least 95% of hits and the correct zone in 80%; and at most 200 ms from the hit to the event. In the demonstration we hit the board by hand and press the admin button, and we watch the seismic graph and the shock wave on the LEDs.

12. M8 LED Animation and Physical Board

Team: Ayşe Poyraz, Hasan Hüseyin Özdemir, Muhib Taha Boy, Umutcan Ocak

12.1 Purpose

M8 builds the physical city board and writes the animation engine that turns the game state into light. The board is 60 × 60 cm. It has 344 addressable LEDs on the roads and in the stores, 3D-printed buildings, and places for the motion sensor and the vibration motors. In the game the board is the only map, so it must be clear to read from every side of the table.

Runs on	Core 1 of the ESP32 for the software; the board itself for the hardware
Technology	Portable C++ animation engine that runs on the ESP32 and on a PC for tests; 344 addressable RGB LEDs; level shifter; 3D printing; plywood base
Requirements	R2 R5 R6 R7 R8 R11

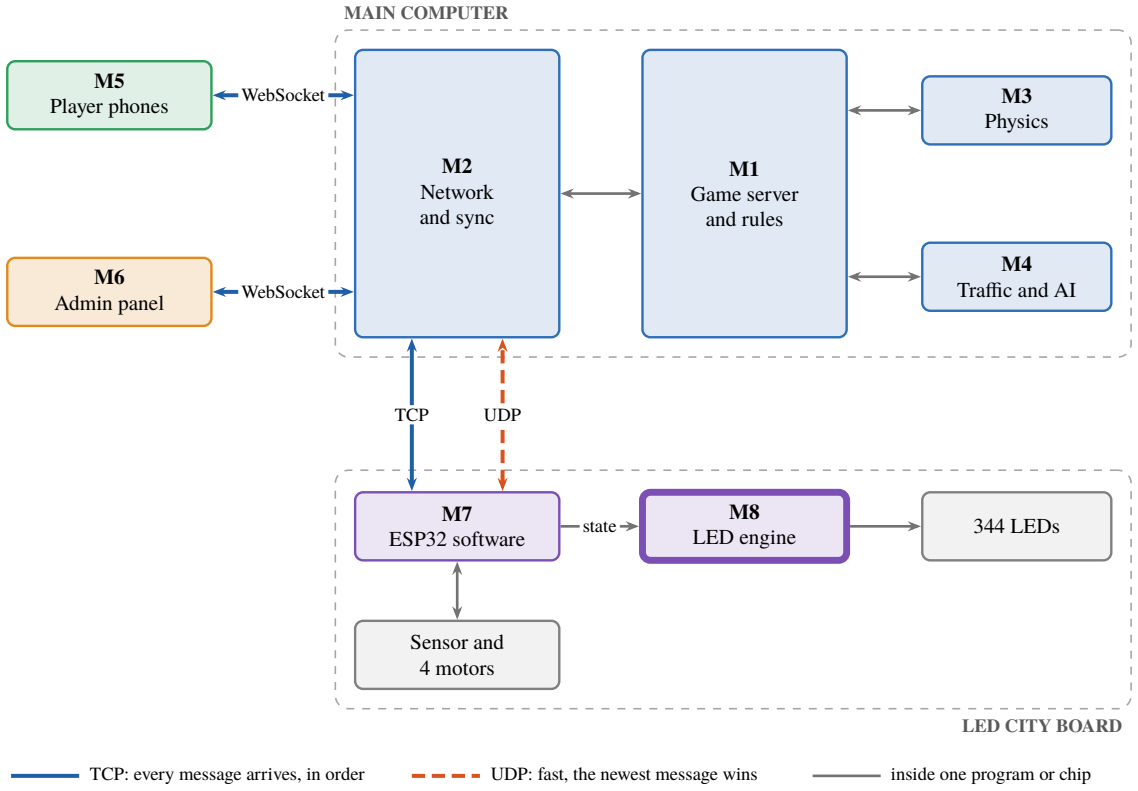


Figure 30: Where M8 sits in the system. M8 gets the state from M7 on the same chip.

12.2 Requirements covered

ID	Requirement	What M8 does for it
R2	Delivery cycle	Delivery points blink in the player's tone; a flash when the package is delivered
R5	No map on the phone	The board is the map: all motorcycles; "Find me" white flash for 1 s
R6	Obstacles	Red patterns by type on the whole road segment
R7	Earthquake	A shock wave from the earthquake centre
R8	Physical LED board	60 × 60 cm, 6 roads in each direction, LEDs on all roads, printed buildings, four stores
R11	Independent development	Demo mode without a server; animations checked as images on a PC

12.3 Inputs and outputs

Dir.	Data	From / to	Content
In	Board state	M7	At most 30 per second: phase, players, delivery points, obstacles, earthquake
In	Config	M7	Player colours, brightness 0.1–1, current limit
In	Link state	M7	Connected, searching or lost
In	City plan	Shared file	Road grid and store places, turned into the LED map when the code is built
Out	Light	Board	60 frames per second
Out	Frame report	M7	Frame time and current estimate, shown to the admin

12.4 Main components

Component	What it does	Related modules
Motion smoother	Moves the motorcycle dots smoothly between two states	M7
LED map	The place of every LED; made from the city plan by a script	M1, city plan
Layers and effects	Five layers and the patterns for obstacles, earthquakes and game phases	—
Post-processing and output	Colour correction, brightness, current limit; drives 5 LED lines at the same time	M7
Demo mode	A built-in scenario that runs without a server	—

12.5 How it works

The board. The base is plywood. On it there are 6 roads in each direction, so the city has 5 × 5 blocks. LED strips with 60 LEDs per metre lie in the roads. A road segment between two crossings is 10 cm long, so it has exactly 6 LED steps. With 336 LEDs on the roads and 2 LEDs in each of the 4 stores, the board has 344 LEDs. The buildings are 3D-printed from the same models as the 3D city on the phone, at a scale of 1 m in the game to 1 mm on the board. Thin printed covers over the roads soften the light. The LEDs need 5 V. The software limits the current to 4 A, so the power supply never gets too hot. A level shifter turns the 3.3 V signal of the ESP32 into a clean 5 V signal.

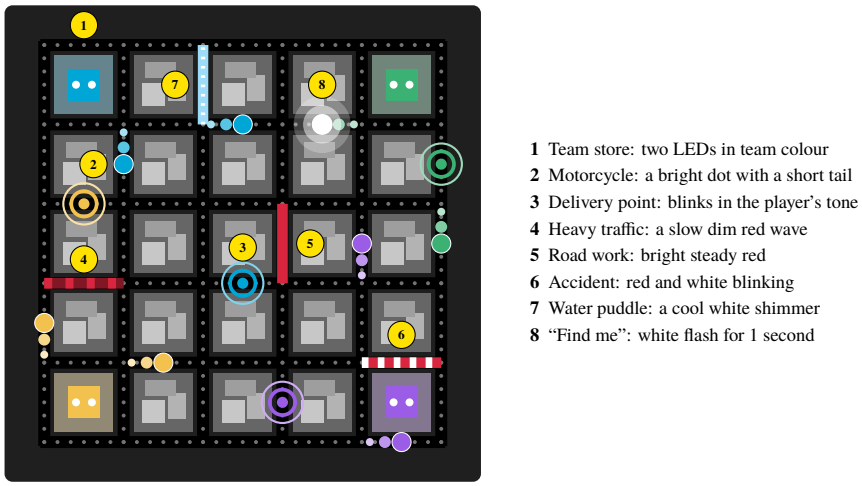


Figure 31: The planned LED board from above, during a game with 8 players.

Colours. There are four teams and each team has one colour: blue, green, yellow and purple. The two players of a team have two tones of the team colour, a light one and a dark one. Red is used only for obstacles, so red always means "this road slows you down". The team colours are far from red, so nobody mixes them up. The pattern of the red tells the type of the obstacle, as the figure below shows.

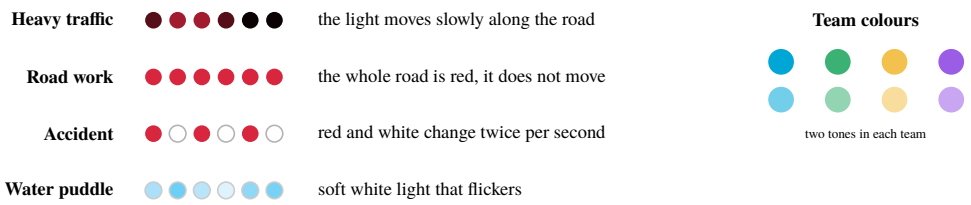


Figure 32: Left: the six LEDs of one road segment for each obstacle type. Right: the team colours.

Layers. The engine draws 60 frames per second on core 1. Each frame is made from five layers, from the bottom to the top: the roads, the obstacles, the delivery points, the motorcycles and a top layer for short effects. A higher layer covers a lower one. The board state comes only 10 to 30 times per second, so the motion smoother moves each motorcycle dot between the last two states. The dot has a short tail, so the direction of motion is easy to see. The top layer draws the "Find me" flash and the earthquake shock wave. After the layers, the engine corrects the colours for the eye, sets the brightness from the config, checks the current and lowers all LEDs a little if the current would be over the limit. Then it sends the frame to the LEDs on 5 lines at the same time, so one frame takes only about 2 ms to send.

Earthquake and link loss. When an earthquake starts, a ring of light grows from the centre of the earthquake over the whole board, like a wave on water, and slowly fades. The roads inside the earthquake radius then turn red for road work. If M7 says that the link is lost, the roads go dark and the store LEDs breathe slowly in amber, so everybody can see that the board is waiting.

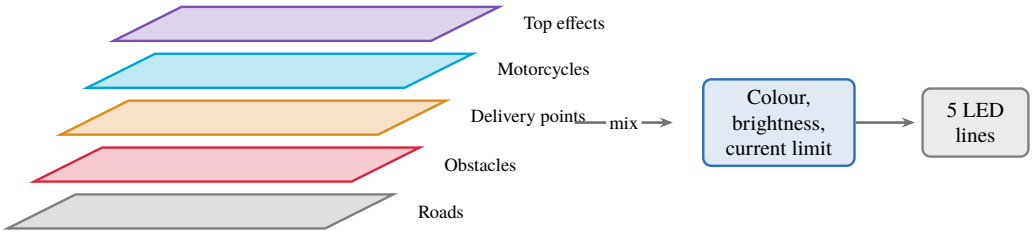


Figure 33: The five layers of one frame and the steps before the LEDs.

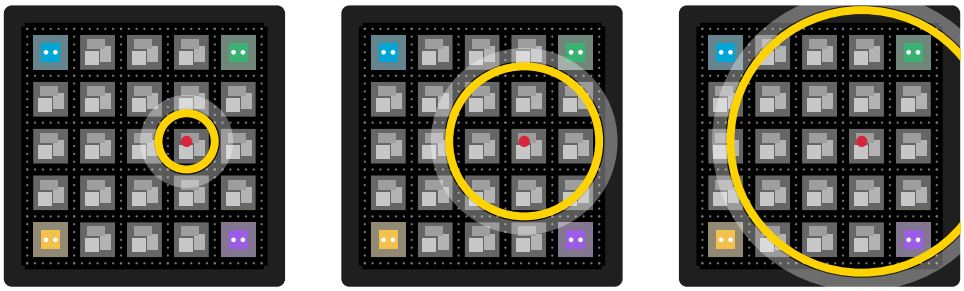


Figure 34: The earthquake shock wave grows from the centre of the hit.

12.6 Testing and demonstration

The engine is plain C++, so it also runs on a PC. There, every animation is drawn as an image and compared with a saved reference image. On the real board, built-in test patterns check the LED order and place, the 8 player colours (three people must name each colour correctly), the current limit, the voltage at the farthest LED, and 30 minutes of heat. In the demonstration the board runs its demo scenario without a server, with all obstacle types and an earthquake.